

Observing Linear Hashing: Optimality and Applications

MU4IN508

Kairui PENG 21219046
Shuoliang YANG 21500629
Feiyang CHU 21208393

Encadré par: Ky Nguyen | Sorbonne Université | 2025-2026

Introduction

Contextualisation, problématique théorique et objectifs du projet de recherche.

Is Linear Hashing Good?

Depuis 1997, une question fondamentale subsiste : l'hachage linéaire est-il suffisant pour garantir une répartition optimale ?

Résultat Théorique (Jaber et al., 2025) :

Une matrice aléatoire sur le corps F_2 est utilisée pour répartir n boules dans n bacs, et la charge maximale L_{max} vérifie:

$$E[L_{max}] = O\left(\frac{\log n}{\log \log n}\right).$$

Ce projet vise à vérifier expérimentalement si une matrice aléatoire sur F_2 permet une répartition optimale des données par rapport au modèle idéal des 'balls and bins'.

Objectifs du Projet

Partie I : Analyse Empirique

- Implémentation et profilage de familles de hachage linéaires sur F_2 .
- Démontrer expérimentalement la charge maximale moyenne optimale.
- Analyse comparative de différentes distributions de clés.

Partie II : Cryptographie

- Prototype de schémas de signature basés sur le hachage linéaire.
- Généralisation sur des corps de caractéristique différente de 2.
- Analyse de la sécurité et de la performance face aux approches standards.

Collaboration Git

Cloner le dépôt

```
git clone URL
```

Créer une branche personnelle

```
git checkout -b dev-<nom>
```

Synchronisation & Mise à jour

```
git checkout main  
git pull origin main  
git checkout dev-<nom>  
git rebase main
```

Développement et soumission

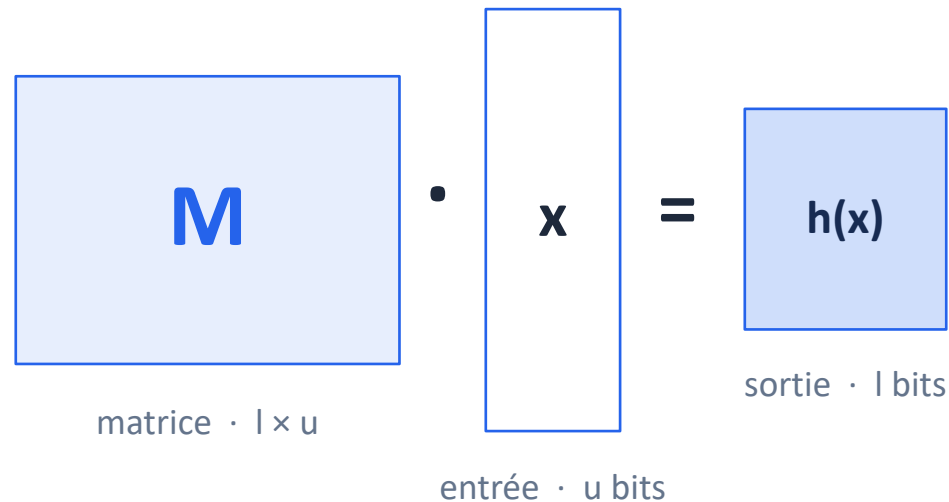
```
git push origin dev-<nom>
```

Partie 1

Hachage linéaire sur F_2 et estimation du max-load

Le hachage linéaire sur F_2

- F_2 : un monde à 2 valeurs $\{0, 1\}$ — toute l'arithmétique se fait modulo 2
 - addition = XOR multiplication = AND
- La fonction de hachage : $h : F_2^u \rightarrow F_2^l$, $h(x) = M \cdot x$
 - elle transforme un vecteur de u bits en un vecteur de l bits



Zoom : du nombre de 1 au bit de sortie

- Le bit y_i = produit scalaire (M_i , x), calculé sur F_2
- Ce produit scalaire est une SOMME de 0 et de 1 — compter les 1 donne cette somme
- Sur F_2 : $1 + 1 = 0$, les 1 s'annulent deux par deux
 - $\rightarrow$ on ne garde que la parité : nombre pair $\rightarrow y_i = 0$, impair $\rightarrow y_i = 1$

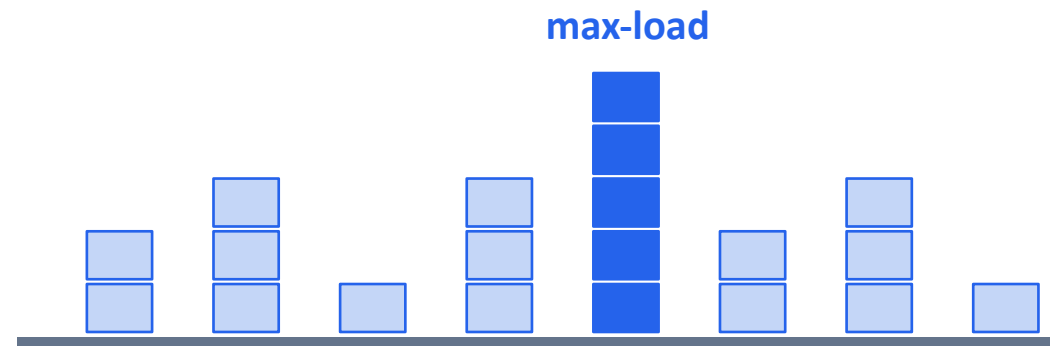


Sur F_2 : $1 + 1 = 0$. Un nombre pair de 1 donne 0, un nombre impair donne 1 — seule la parité compte.

L'expérience : le max-load

- On lance m balles (l'ensemble S) dans $n = 2^l$ paniers, à travers h
- $\text{max-load} = M(S, h) =$ la charge du panier le plus rempli
- But : estimer $P[\text{max-load} \geq r \cdot \log n / \log \log n] \approx 1 / r^2$
 - r : paramètre qui règle le seuil — on le fait varier pour tracer la courbe

m balles $\rightarrow h \rightarrow n = 2^l$ paniers



$\text{max-load} =$ la charge (nombre de balles) du panier le plus rempli

Le point de départ : 100 % Python

sampling.py

Échantillonnage

```
x =  
rng.getrandbits(u)
```

Génère un vecteur
aléatoire $x \in \mathbb{F}_2^u$

Un entier Python sert de
masque de u bits

linear_f2.py

Hachage linéaire

```
(M_i & x).bit_count()  
& 1
```

$h(x) = M \cdot x$ sur $\mathbb{F}_2$, ligne
par ligne

Produit scalaire $\mathbb{F}_2$ via
opérations bit à bit

maxload.py

Comptage max-load

```
defaultdict(int)  
[compteur exact]
```

Compte les éléments
par panier

Un compteur par panier →
max-load = max(counts)

× GOULOT 1 — TEMPS (CPU)

Surcoût de l'interpréteur Python ;

`bit_count()` appelé $l \times m$ fois par trial

× GOULOT 2 — ESPACE (MÉMOIRE)

Les 2^b paniers explosent en mémoire ;

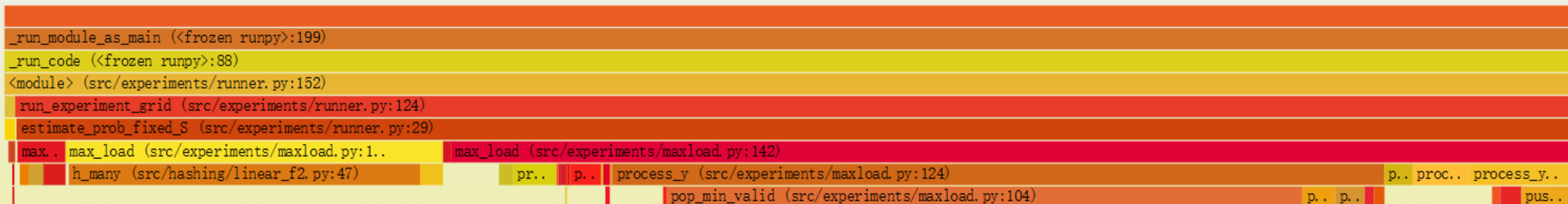
pour $l = 100$, cela fait **2^{100} paniers**

Problème de temps (CPU)

```
$ py-spy record -o profile.svg -- python -m src.experiments.runner
```

```
py-spy record -o profile.svg -- python -m src.experiments.runner
```

Search



✗ Constat

100 % du CPU dans **max_load()**

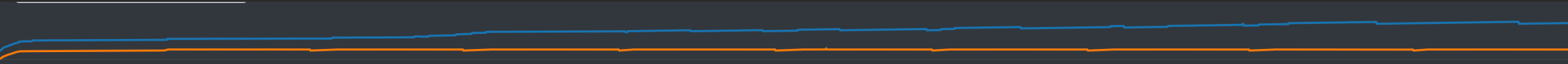
Deux hot-paths à parts égales

Détail des hot-paths

1. **h_many** (linear_f2.py) — hachage F_2
2. **process_y** + **pop_min_valid** (maxload.py) — comptage + tas

Problème de mémoire

```
$ memray run -o memray.bin -m src.experiments.runner
```



```
<root>
runpy.run_module(args.script, run_name="__main__", alter_sys=True)
results = {}
from src.experiments.maxload import Maxload

if use_batch:
    h_many = getattr(self.h, "h_many", None)
    for x in iterable:
        rng = random.Random(seed)
        return _SAMPLERS[dist](u=u, rng=rng, **...)
        return rng.getrandbits(u)
```

✗ Constat

Allocations concentrées dans **Maxload**
+ boucle d'échantillonnage (**getrandbits**)

Cause

Comptage exact : **defaultdict(int)**
un compteur par panier, soit **2^b entrées**

Problème : « ça se compte en années »

Mesure de référence ($l = 15$)

$u = 200$, $m = 49\,152$ balles

1,38 ms par trial → 5 000 trials en **6,89 s**

À $l = 100$: $m = 1,4 \cdot 2^{100}$

$\approx 1,77 \times 10^{30}$ balles

soit **$3,6 \times 10^{25}$** fois plus qu'à $l = 15$



Coût d'un seul trial à $l = 100$:

$1,38 \text{ ms} \times 3,6 \times 10^{25} \approx \mathbf{1,6 \times 10^{15} \text{ années}}$ — **totalelement infaisable**

La limite pratique : $l = 20$, $m = 1,4 \cdot 2^{20} \approx 1,47$ million

$\approx 41 \text{ ms}$ par trial → 2 000 trials $\approx \mathbf{82 \text{ s}}$, **tout juste acceptable**

Backend C++ : architecture

Remplacement 1:1 des trois modules Python par du C++ natif

sampling.py → C++

Échantillonnage

samplers.hpp

Optimisations :

- Random : **mt19937_64** natif
- Remplissage direct des **uint64[]**
- Zéro allocation dans la boucle

linear_f2.py → C++

Hachage F_2

linear_hash.hpp/cpp

Optimisations :

- Matrice packée en **uint64 blocks**
- Produit F_2 : **popcountll(row & x) & 1**
- Variante **hash_u32** pour $l \leq 32$

maxload.py → C++

Comptage max-load

space_saving.hpp

trial_maxload.cpp

Optimisations :

- Space-Saving C++ : **uint32** compteurs
- Tas paresseux avec **version**
- **fingerprint64** : SplitMix64 + XOR
- clé uint64 → pas de **defaultdict**

Résoudre le goulot mémoire : Space-Saving

✗ Problème

```
count = defaultdict(int)
for x in S:
    count[h(x)] += 1
max_load =
max(count.values())
```

Comptage exact :

un compteur par panier

Mémoire = **$O(2^l)$**

$l = 20 \rightarrow 10^6$ entrées ✓

$l = 30 \rightarrow 10^9$ entrées ⚠

$l = 100 \rightarrow 2^{100}$ ✗

✓ Space-Saving

Idée : on ne cherche que le **max**
→ inutile de compter **tous** les paniers
Garder seulement les **top-k** candidats

Fonctionnement :

Si $y \in \text{table}$: incrémenter

Si $|\text{table}| < k$: insérer

Sinon : remplacer le **minimum**

Mémoire = **$O(k)$** ($k = 50\,000$)

Temps = **$O(m \log k)$** par trial

⚡ Version C++

Tas paresseux (lazy heap) :

- Chaque entry a un **numéro de version**
- **push** systématique, **pop** filtré
- Évite de reconstruire le tas

fingerprint64 :

- $h(x)$ multi-block → **1 seul uint64**
- SplitMix64 + XOR chaîné
- Élimine **defaultdict** → **unordered_map**

Compteurs : **uint32**

(4 octets vs 28 en Python)

Parallélisation + intégration Python

Parallélisation multi-thread

`parallel_trials.hpp`

Observation : chaque trial est indépendant

→ parallélisme trivial sur les trials

- **`std::thread`** × N (N = `hardware_concurrency`)
- **`atomic<size_t>`** pour distribution des tâches
- Modèle **work-stealing** : `idx.fetch_add(1)`
- Chaque thread exécute un trial complet :
`sample` → `hash` → `SpaceSaving` → `max_count`
- **Zéro synchronisation inter-thread**
(pas de mutex, pas de section critique)

Module pybind11 : fasthash

`bindings.cpp`

- **LinearHash** : `hash_int` / `hash_many_int`
- **`run_trials_maxload`** : entrée unique all-in-C++
- **`gil_scoped_release`** pendant le calcul C++
- Conversion Python `int` ↔ `uint64[]` transparente

Appel depuis Python

```
import fasthash

results = fasthash.run_trials_maxload(
    u=3000, l=30, m=m,
    dist="uniform",
    seeds_S=seeds_S, seeds_h=seeds_h,
    k=50000, num_threads=10)
```


Comparaison : C++ Parallel vs Python

```
(.venv313) → Observing-Linear-Hashing git:(dev-Kairui) x python -m scripts.compare --u 50 --l 10 --n 200000
=== Benchmark: u=50, l=10, n=200000, seed=12345 ===
[PY single] times: 0.183562s, 0.184968s, 0.184194s, 0.183403s, 0.185448s
[PY single] median: 921.0 ns/op => 1.086 M ops/s
[PY batch ] times: 0.189104s, 0.188838s, 0.188471s, 0.187876s, 0.188472s
[PY batch ] median: 942.4 ns/op => 1.061 M ops/s
[C++ single] times: 0.044758s, 0.044619s, 0.044416s, 0.044350s, 0.044308s
[C++ single] median: 222.1 ns/op => 4.503 M ops/s
[C++ batch ] times: 0.027142s, 0.026891s, 0.026948s, 0.026894s, 0.027015s
[C++ batch ] median: 134.7 ns/op => 7.422 M ops/s

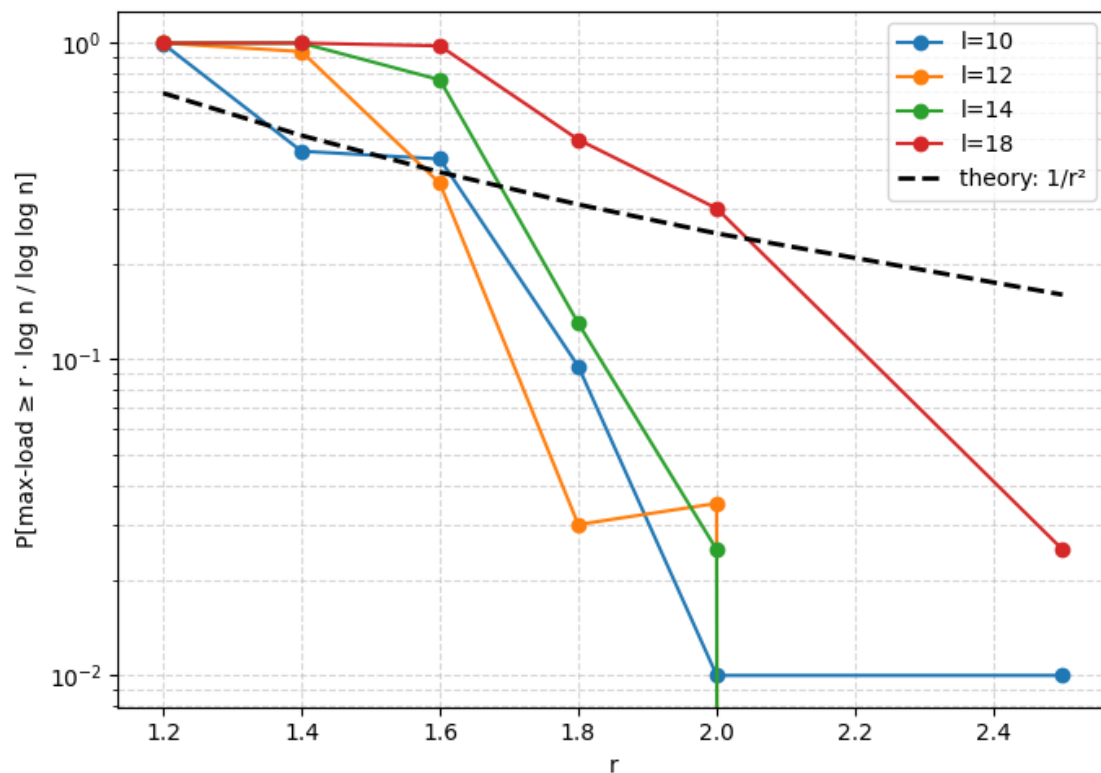
=== Speedups ===
C++ single over PY single: 4.15x
C++ batch over PY single: 6.84x
C++ single over PY batch : 4.24x
C++ batch over PY batch : 6.99x

=== C++ Multi-threaded Trials ===
  threads= 1 median=0.0253s per_trial=0.13ms
  threads= 4 median=0.0072s per_trial=0.04ms
  threads= 8 median=0.0050s per_trial=0.03ms
  threads=10 median=0.0044s per_trial=0.02ms
```

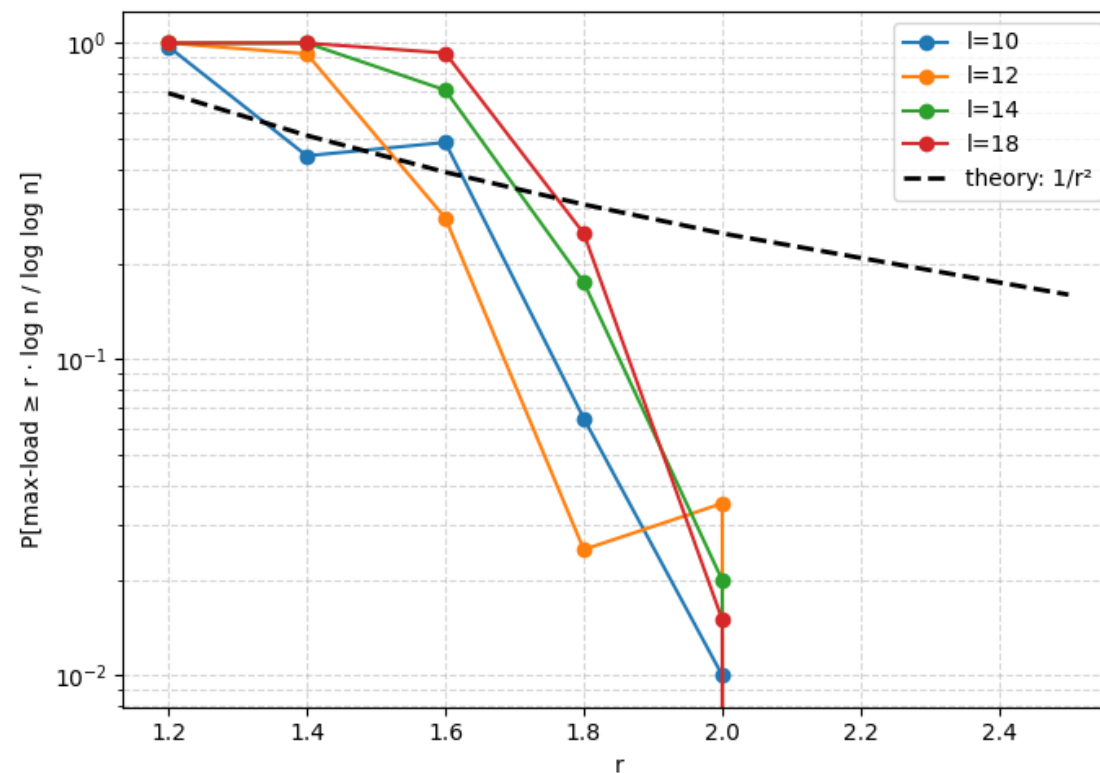
Résultat du Partie I

Pour tout $(S \subseteq F_u^2)$ de taille m et $r \geq 6$, on a :

$$\left[\Pr_{h \sim \mathcal{H}} \left[M(S, h) \geq r \cdot \text{opt}(m, n) \right] \leq \frac{49}{(r-2)^2} \right]$$



u (la dimension des données d'entrée)=20



u (la dimension des données d'entrée)=32

Partie 2 | MuSig2-H

Multi-Signatures basée sur une fonction de hachage linéaire

Évolution de la sécurité des multi-signatures

MuSig2

$$F(x) = x \cdot G$$

$$\mathbb{Z}_p \rightarrow E(F_p)$$

✗ **bijectif**

⚠ **nécessite OMDL**

- n signataires $\rightarrow$ 1 signature agrégée
- Noyau : $x \rightarrow x \cdot G$ (scalaire $\times$ point de base), une fonction de hachage linéaire
- Sécurité repose sur **OMDL (One-More Discrete Log)**, plus fort que l'hypothèse **DL (Discrete Log)** standard

MuSig2-H (TZ23)

$$F(x_1, x_2) = x_1 \cdot G + x_2 \cdot Z$$

$$\mathbb{Z}_{p^2} \rightarrow E(F_p)$$

✓ **non-injectif** (Tiroirs)

✓ hypothèse DL standard suffit

- Remplacer $x \cdot G$ par une LHF générale **F**
- Exigence : F est **non-injectif** (collisions existent)
- $x \cdot G$ est bijectif $\rightarrow$ **ne satisfait PAS cette condition**
- Engagement de Pedersen : espace d'entrée $\mathbb{Z}_{p^2}$
- Espace de sortie $E(F_p) \rightarrow$ naturellement non-injectif

MuSig2-H : 8 algorithmes (TZ23 Fig. 4)

#	Algorithme	Exécuteur	Fonction
1	Setup	Système	Générer les paramètres publics (p, E, G, Z)
2	KeyGen	Chaque signataire	Générer la paire de clés : $sk \leftarrow Z_q, pk = sk \cdot G$
3	KeyAgg	Quiconque	Entrée : liste L de clés publiques $\rightarrow$ clé agrégée $apk = \sum H_{agg}(L, pk_i) \cdot pk_i$
4	PreSign	Chaque signataire	Générer 4 paires de nonces $r_j \in Z_{q^2}, R_j = F(r_j)$
5	PreAgg	Quiconque	Agréger les nonces : $R_j = \sum_i R_{\{i,j\}}$
6	Sign	Chaque signataire	Calculer la signature partielle (R, s_i) avec sk, r_j, m
7	SignAgg	Quiconque	Agréger les signatures partielles : $s = \sum s_i, \sigma = (R, s)$
8	Ver	Quiconque	Vérifier $F(s) = R + H_{sig}(apk, R, m) \cdot apk$

Flux d'exécution : 2 points de synchronisation

- | | | | |
|---|--------------------|---|------------------------|
| ① | KeyGen × n | ← parallèle : chaque signataire génère ses clés indépendamment | // parallèle |
| ② | KeyAgg | ← séquentiel : collecter toutes les clés publiques, puis agréger | |
| ③ | PreSign × n | ← parallèle : chaque signataire génère 4 paires de nonces (hors-ligne) | // parallèle |
| ④ | PreAgg | ← séquentiel : agréger les nonces | Point de sync 1 |
| ⑤ | Sign × n | ← parallèle : chaque signataire calcule sa signature partielle (en-ligne) | // parallèle |
| ⑥ | SignAgg | ← séquentiel : agréger les signatures partielles | Point de sync 2 |
| ⑦ | Ver | ← séquentiel : vérification finale | |

Choix techniques : bibliothèque et courbe

Pourquoi SageMath ?

Besoins :

$P + Q$	addition de points
$k \cdot P$	multiplication scalaire
$a \bmod p$	arithmétique sur $\mathbb{Z}_p$

- Courbes elliptiques + corps finis **natifs**
- Arithmétique **exacte** sur $\mathbb{Z}_p$ (pas de flottants)
- Code $\approx$ formules du papier

Pourquoi Curve25519 ?

```
p = 2255 - 19
E : y2 = x3 + 486662x2 + x (Montgomery)
cofacteur = 8, q = |E| / 8
q = 2252 + C (premier)
```

- Courbe **standardisée** (TLS, Signal, SSH)
- Sécurité ~ 128 bits
- Satisfait les conditions de taille :
 - $|S| = |Z_q| = q \approx 2^{252}$
 - $|D| = q^2, |R| = q$
 - tous $\geq 2^{128}$ (**Déf. 1, TZ23 §3.1**)
- G : point de base standard (x=9)
- Z : générateur indépendant (hash-to-curve)

Implémentation et tests en Python

Architecture en couches

curve.py

81 lignes

Courbe Curve25519 (SageMath) : F_p , E , G , Z , q

lhf.py

47 lignes

Fonction de hachage linéaire : $F(x_1, x_2) = x_1 \cdot G + x_2 \cdot Z$

musig2h.py

251 lignes

8 algorithmes MuSig2-H + 3 fonctions de hachage (SHA-256)

signer.py

100 lignes

Machine à états : KeyGen $\rightarrow$ PreSign $\rightarrow$ Sign

parallel_signing.py 151 lignes

Coordinateur du protocole : 7 étapes + chronométrage

82 tests (pytest)

Fichier

#

Couverture

test_curve.py

21

propriétés de la courbe elliptique

test_lhf.py

14

linéarité, surjectivité, non-injectivité

test_musig2h.py

24

correction des 8 algorithmes

test_signer.py

14

machine à états, réutilisation de nonces

test_parallel.py

9

protocole bout-en-bout (n=2..10)

3 niveaux de test :

Unitaire (curve, lhf)

$\rightarrow$ **Intégration** (musig2h, signer)

$\rightarrow$ **Bout-en-bout** (parallel)

Problème : PARI n'est pas thread-safe

Chaîne d'appels → Segmentation fault

`Signer.__init__()`

→ `musig2h.keygen()`

→ `lhf.F_key(sk)`

→ `curve.scalar_mult(sk, G)`

→ `Integer(k) * P`

→ `pari.ellmul(E, self, k)`

→ `cypari2 [libère le GIL]`

→ PARI C code

→ pile globale PARI

⚡ **Segfault (100%)**

9/9 tests → 100% crash

`ellmul` est trop long → conflit garanti

Pile globale PARI

Thread 1: [GIL] → `cypari2` → **[libère GIL]** → PARI C

Thread 2: [GIL] → `cypari2` → **[libère GIL]** → PARI C

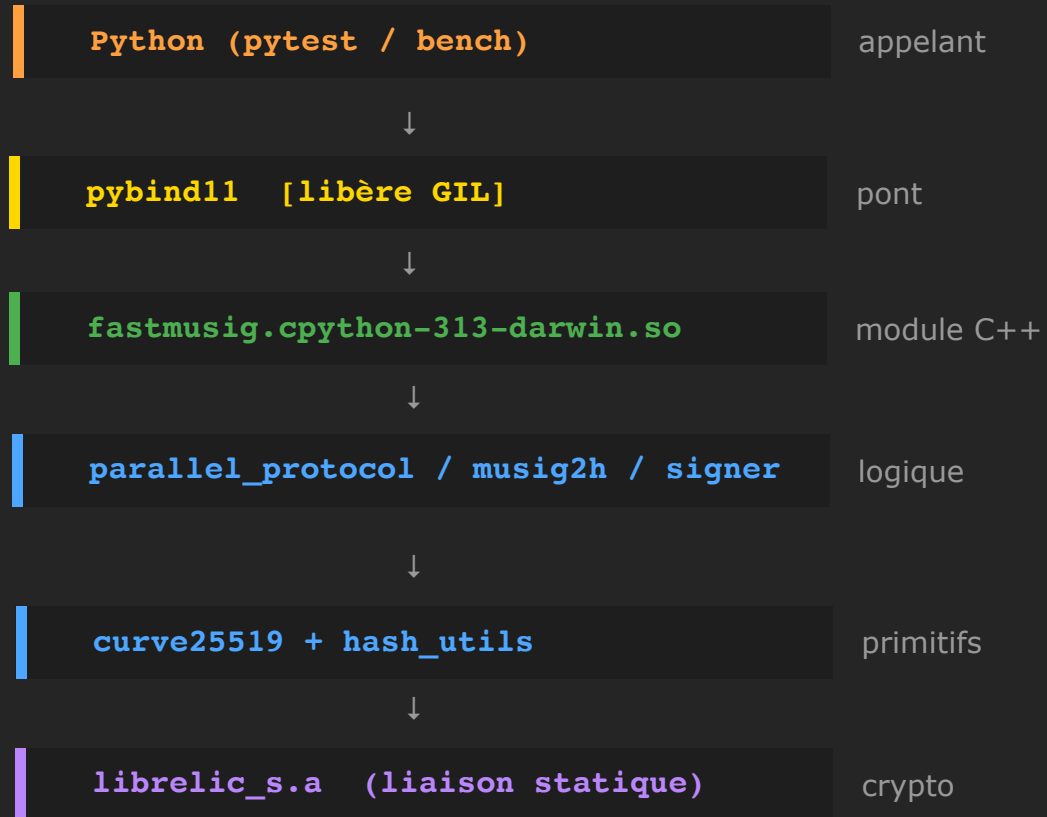
Les deux threads écrivent sur la **même pile** sans aucun verrou → **écrasement mutuel**

Alternatives évaluées

Approche	Résultat	Raison
ThreadPool	Segfault	pile PARI partagée
ProcessPool	Impossible	pickle échoue (Sage obj)
asyncio	Inutile	CPU-bound, pas I/O
Séquentiel	Adopté	structure parallèle conservée
C++ / RELIC	Solution	thread-safe par conception

C++ RELIC : architecture

Pile technologique



4 phases de développement

Phase 1 Courbe + hachage

- Point / Scalar (RELIC)
- Montgomery ↔ Weierstrass
- SHA-256 (OpenSSL, séparation de domaine)

Phase 2 8 algorithmes + Signer

- Traduction fonction par fonction depuis Python
- Signer : machine à états (KeyGen→PreSign→Sign)
- run_protocol_sequential

Phase 3 Parallélisation

- RelicThreadPool (std::thread)
- core_init() par thread + paramètres courbe
- run_protocol_parallel (barrière par phase)

Phase 4 Benchmarks

- Python vs C++ (1/2/4/8/12 threads)
- n = 2..100 signataires
- Accélération mesurée : 9.6×

Pont pybind11

pybind11 : pont C++ ↔ Python

- Module **fastmusig** : module pybind11 compilé à partir du code C++
- **GIL libéré** pendant le calcul C++
- Conversion automatique :
 - bytes ↔ Point, int ↔ Scalar
- Même interface que les modules Python
 - Tests Python fonctionnent avec les deux backends (76/76 **tous passés** ✓)
- **Exports :**
 - constantes (G, Z, q), 8 algorithmes,
 - Signer, run_protocol_sequential,
 - run_protocol_parallel

méthodes d'appel

```
import fastmusig as fm

fm.setup()           # constantes
fm.keygen(seed)      # KeyPair
fm.presign(seed)     # 4 nonces
fm.sign(...)         # signature
fm.ver(apk, m, sig)  # bool
...
```

Architecture parallèle : RelicThreadPool

Flux du protocole parallèle

KeyGen × n

[pool.run_
phase]

chaque thread : core_init() +
keygen()

KeyAgg

thread principal : agréger les
clés

PreSign × n

[pool.run_
phase]

chaque thread : 4 paires de nonces **Sync 1**

PreAgg

thread principal : agréger les
nonces

Sign × n

[pool.run_
phase]

chaque thread : signature
partielle **Sync 2**

SignAgg

thread principal : $s = \sum s_i$

Ver

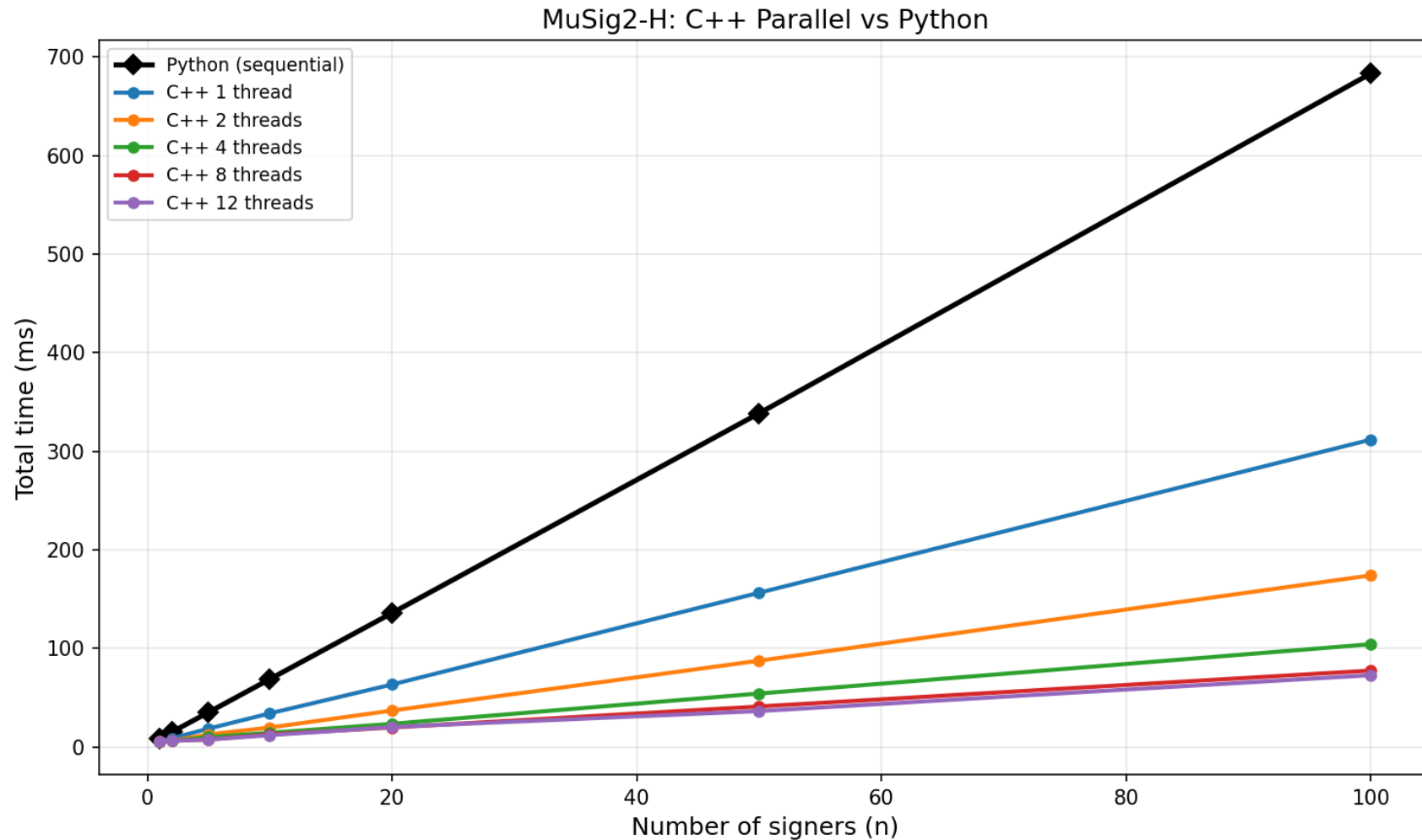
thread principal : $F(s) = R +$
 $c \cdot apk$

RelicThreadPool

```
// Construction
pool(n_threads)
    chaque worker:
        core_init() // RELIC
        ep_param_set(CURVE_25519)

// Exécution d'une phase
pool.run_phase(n, fn)
    dispatch n tâches
    barrière : attend tous
```

Résultat du Partie II : C++ vs Python

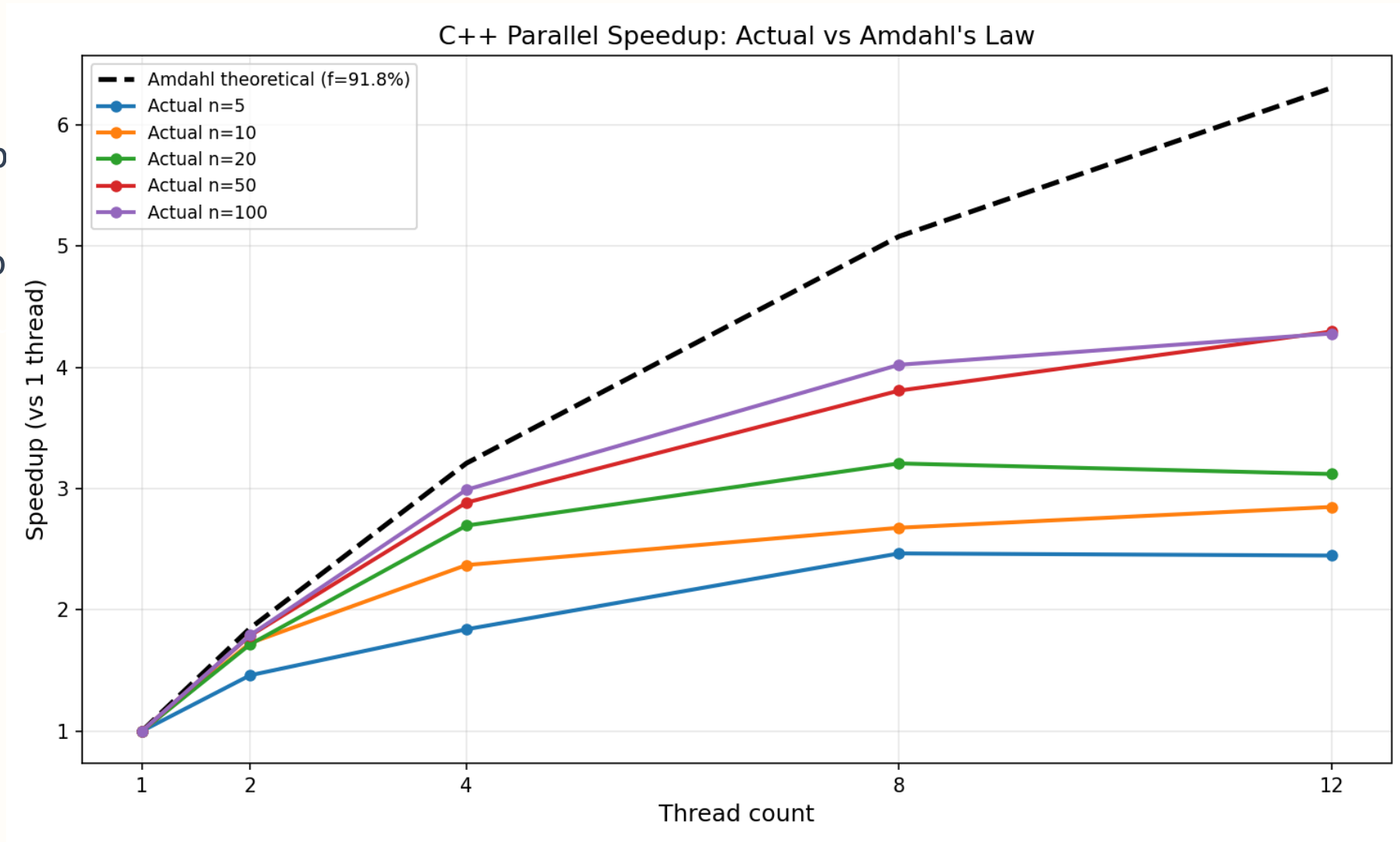


C++ Parallel vs Loi d'Amdahl

Loi d'Amdahl présente S_p

Si P désigne la proportion
par :

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$



Conclusion

Conclusion

- Dans la première partie du projet, nous modélisons les fonctions de hachage sur $\mathbb{F}_2$, essayons de démontrer expérimentalement que la charge maximale (max-load) correspond bien aux limites théoriques, et optimiser les performances de Python via une réécriture en C++ pour obtenir une accélération d'un facteur de plusieurs dizaines.
- Dans la seconde partie du projet, nous avons étendu le hachage linéaire au domaine de la cryptographie en implémentant le schéma de multi-signature MuSig2-H sur la courbe elliptique Curve25519.
De plus, l'utilisation combinée de la bibliothèque C++ RELIC et du multithreading a permis de contourner les limites de sécurité des threads de Python, tout en réalisant une accélération approchant la limite théorique de la loi d'Amdahl.

Merci!

Kairui PENG | Shuoliang YANG | Feiyang CHU
Encadré par: Ky Nguyen | Sorbonne Université | 2025-2026